

Student Performance Prediction Project

Craig Reynolds

Machine Learning Regression

Python | Scikit-Learn | Pandas

Introduction:

In this project I will be exploring a dataset composed of math students with the goal of training a regression model that can predict their final grades. This predictive ability would be very useful because it would allow educators to identify at risk students and offer extra help before they begin to fail. Since the goal is early intervention, I will be excluding the Term 1 and Term 2 grades (G1 & G2) when attempting to predict the Term 3 grade (G3). Although the absences for each term would not be available early in the year, I will include them in this project because they provide an opportunity for feature engineering. Since prior term grades are the strongest predictors of the target, modeling without them will prove to be a very challenging endeavor.

Dataset Information

This project uses a student performance dataset originating from the UCI Machine Learning Repository. The dataset contains demographic, social, and academic information for Portuguese secondary school students.

Tools and Libraries Used

-Python -Pandas -NumPy -Scikit-Learn -Matplotlib -Seaborn

Summary of Dataset Features

Feature	Description
school	Student's school (GP = Gabriel Pereira, MS = Mousinho da Silveira)
sex	Student's sex
age	Student's age
address	Home address type (U = urban, R = rural)
famsize	Family size (LE3 = less than or equal to 3, GT3 = greater than 3)
Pstatus	Parent cohabitation status (T = together, A = apart)
Medu	Mother's education level (0–4 scale)
Fedu	Father's education level (0–4 scale)
Mjob	Mother's occupation
Fjob	Father's occupation
reason	Reason for choosing the school
guardian	Student's guardian
traveltime	Home-to-school travel time (1–4 scale)
studytime	Weekly study time (1–4 scale)
failures	Number of previous class failures
schoolsup	Extra educational support (yes , no)
famsup	Family educational support (yes , no)
paid	Extra paid classes (yes , no)
activities	Participation in extracurricular activities (yes , no)
nursery	Attended nursery school (yes , no)
higher	Desire to pursue higher education (yes , no)
internet	Internet access at home (yes , no)
romantic	In a romantic relationship (yes , no)
famrel	Quality of family relationships (1–5 scale)
freetime	Free time after school (1–5 scale)
goout	Frequency of going out with friends (1–5 scale)
Dalc	Workday alcohol consumption (1–5 scale)
Walc	Weekend alcohol consumption (1–5 scale)
health	Current health status (1–5 scale)
absences_G1	Number of absences during Term 1
absences_G2	Number of absences during Term 2
absences_G3	Number of absences during Term 3
G1	First Term grade
G2	Second Term grade
G3	Final grade (target variable)
total_absences	Engineered feature representing total absences across all terms

First Steps

I will begin this project by loading the csv file, identifying the target column I want to predict and using train_test_split to create a training set and test set. It is important to split the training set and test set as early as possible to avoid data leakage.

```
In [4]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split

#load csv as a dataframe
studentmath = pd.read_csv('student-mat.csv')

#Create variables for target and features
y = studentmath['G3']
X = studentmath.drop(columns = ['G3'])

#Perform the splitting of data into training set and test set
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Dataset Exploration

After loading the CSV file and identifying the target feature I want to predict, the next step is to explore the dataset to gain some understanding of the structure by taking a look at the shape, column names, data types and number of missing values.

```
In [5]: #View the number of students(rows) and number of features(columns) in the dataset
print('\n')
print('Shape:', studentmath.shape)
print('\n')

#View the datatypes, missing values and percent of missing values for each column
summary = pd.DataFrame({
    "Dtype": studentmath.dtypes,
    "Number Missing": studentmath.isna().sum(),
    "PCT Missing": (studentmath.isna().mean() * 100).round(2) })

print(summary)
print('\n')
```

Shape: (395, 35)

	Dtype	Number Missing	PCT Missing
school	object	0	0.00
sex	object	0	0.00
age	float64	12	3.04
address	object	0	0.00
famsize	object	0	0.00
Pstatus	object	0	0.00
Medu	int64	0	0.00
Fedu	int64	0	0.00
Mjob	object	0	0.00
Fjob	object	0	0.00
reason	object	0	0.00
guardian	object	0	0.00
traveltime	int64	0	0.00
studytime	int64	0	0.00
failures	int64	0	0.00
schoolsup	object	0	0.00
famsup	object	0	0.00
paid	object	0	0.00
activities	object	0	0.00
nursery	object	0	0.00
higher	object	0	0.00
internet	object	0	0.00
romantic	object	0	0.00
famrel	int64	0	0.00
freetime	int64	0	0.00
goout	int64	0	0.00
Dalc	int64	0	0.00
Walc	int64	0	0.00
health	int64	0	0.00
absences_G1	float64	13	3.29
absences_G2	float64	9	2.28
absences_G3	float64	14	3.54
G1	int64	0	0.00
G2	int64	0	0.00
G3	int64	0	0.00

Feature Exploration

The next step is to begin exploring the various features to see how they relate to the target, G3. This will provide an idea of which features to use when modeling and which do not appear to provide useful signal. This is a relatively clean dataset so data cleaning will be minimal in this project. My first exploration will be creating a heatmap of feature correlations. Continuous numeric features will be evaluated using Pearson correlation and ordinal features will be evaluated using Spearman correlation.

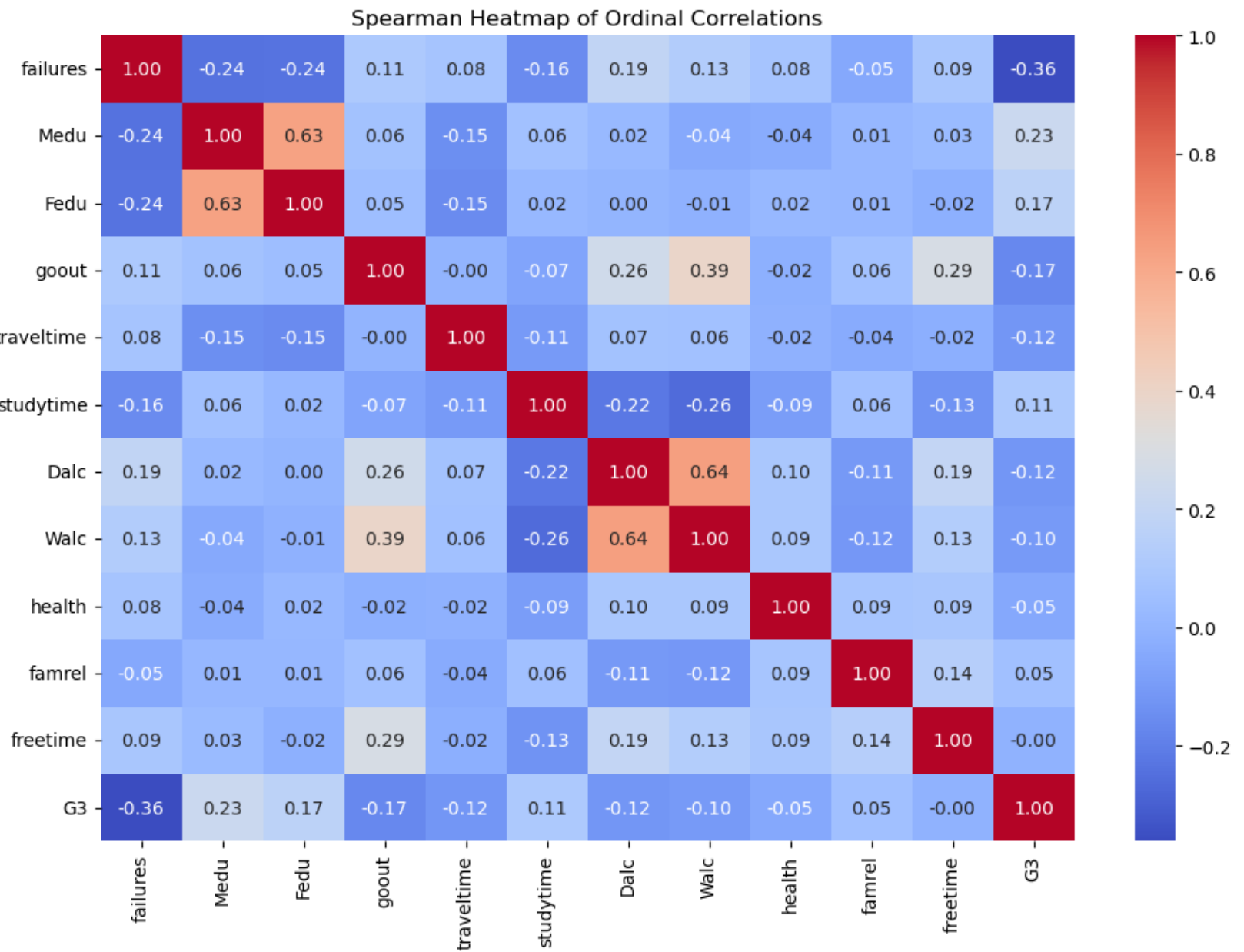
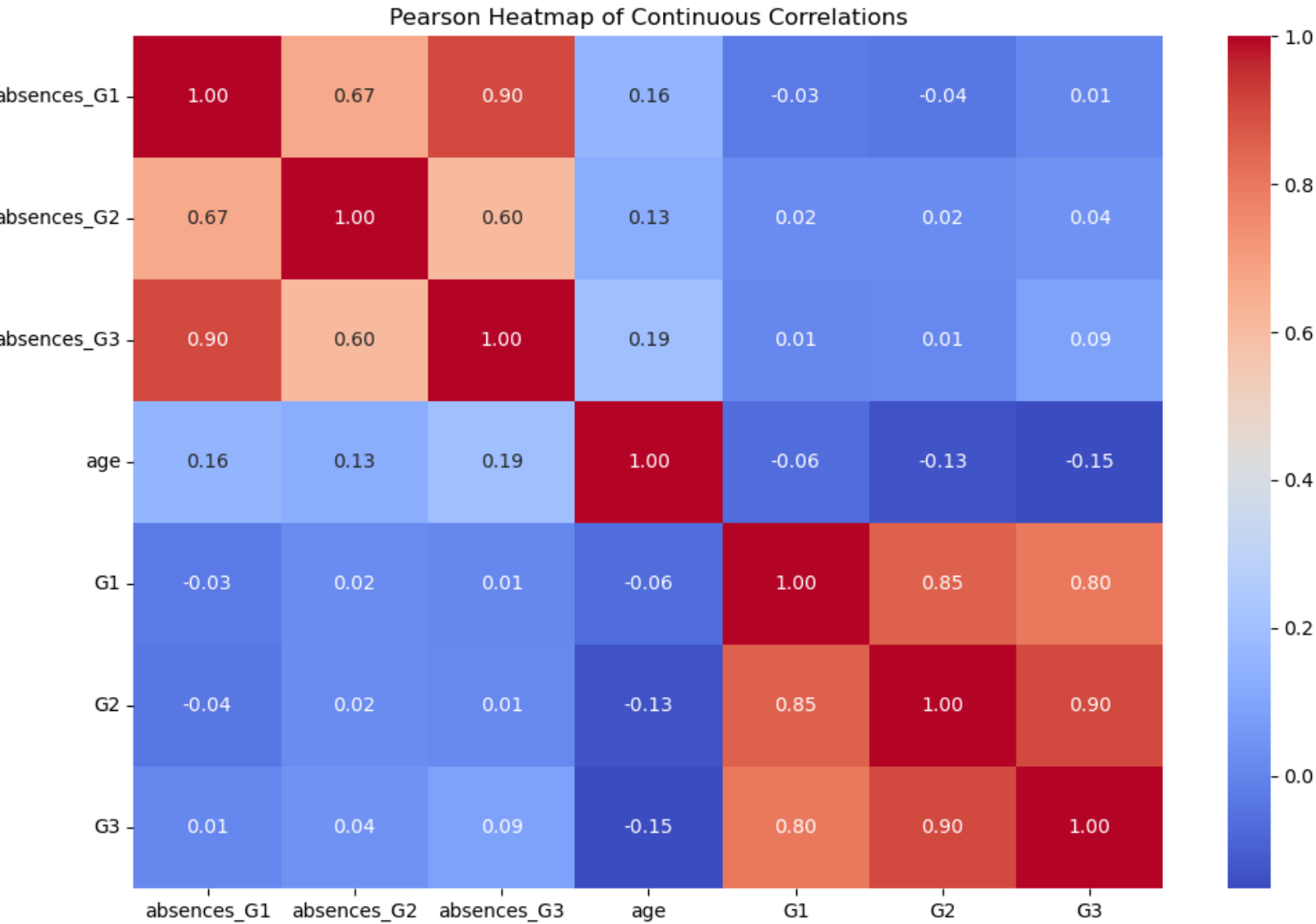
```
In [317]: import matplotlib.pyplot as plt
import seaborn as sns

print('\n')

#Create a list of the continuous and ordinal features based off the codebook
continuous = ['absences_G1', 'absences_G2', 'absences_G3','age', 'G1', 'G2', 'G3']
ordinal = ['failures', 'Medu', 'Fedu', 'goout', 'traveltime', 'studytime', 'Dalc', 'Walc', 'health', 'famrel', 'freetime', 'G3']
```

```
#Plot a heatmap of the continuous feature correlations
plt.figure(figsize = (12,8))
sns.heatmap(studentmath[continuous].corr(method='pearson'), annot=True, fmt=".2f", cmap="coolwarm")
plt.title("Pearson Heatmap of Continuous Correlations")
plt.show()
print('\n')

#Plot a heatmap of the ordinal feature correlations
plt.figure(figsize = (12,8))
sns.heatmap(studentmath[ordinal].corr(method='spearman'), annot=True, fmt=".2f", cmap="coolwarm")
plt.title("Spearman Heatmap of Ordinal Correlations")
plt.show()
print('\n')
```



Heatmap Interpretations

The features that have the strongest linear relationships with G3 are: previous failures (failures), parental education level (Medu, Fedu), age (age), how often they go out (goout), their distance from the school (traveltime), daily alcohol consumption (Dalc) and how often they study (studytime). Aside from G1 and G2 which will be excluded from modeling, all features appear to have only weak relationships with G3. This will make modeling a challenging task.

Exploration of failures feature

Since failures has the strongest correlation with G3, I will explore this feature first by making a table and boxplot that illustrates the average final grade of students in relation to how many failures they had.

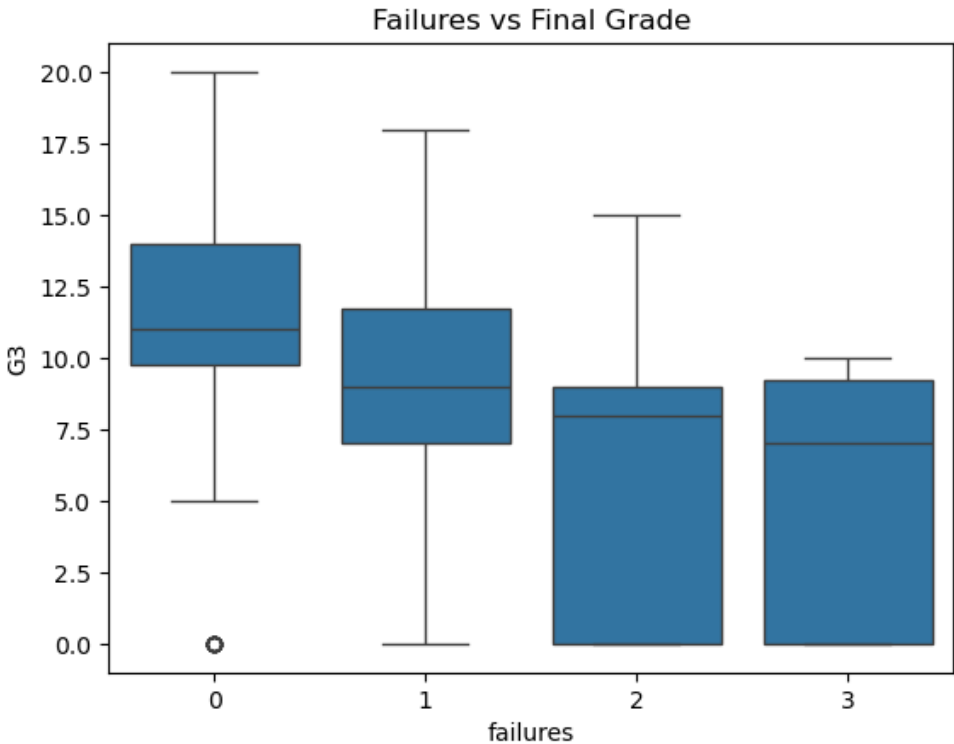
In [247]

```
#Explore failures which has the strongest correlation at -0.36. A weak negative correlation.
print('\n')

#Create a table of avg final grade grouped by number of failures
result = studentmath.groupby('failures')['G3'].mean().round(2).reset_index()
result.columns = ['Failures', 'Average G3']
print(result.to_string(index=False))
print(' \n')

#Create a boxplot of failures vs G3
sns.boxplot(x='failures', y='G3', data=studentmath)
plt.title("Failures vs Final Grade")
plt.show()
print('\n')
```

Failures	Average G3
0	11.25
1	8.12
2	6.24
3	5.69



Interpretation of Failures

Failures appears to be a relatively useful feature with a stable, consistent trend. More failures results in lower grades, which also makes logical sense. Failures will be included in modeling.

Exploration of Parental Education Features

Next, I will explore the parental education features, Medu and Fedu with some tables and bar plots illustrating their structure and trends.

In [318]

```
print('\n')
#Create tables of avg final grade grouped by parental education levels
Medu = studentmath.groupby('Medu')['G3'].mean().round(2).reset_index()
Medu.columns = ['Medu', 'Average G3']

Fedu = studentmath.groupby('Fedu')['G3'].mean().round(2).reset_index()
Fedu.columns = ['Fedu', 'Average G3']

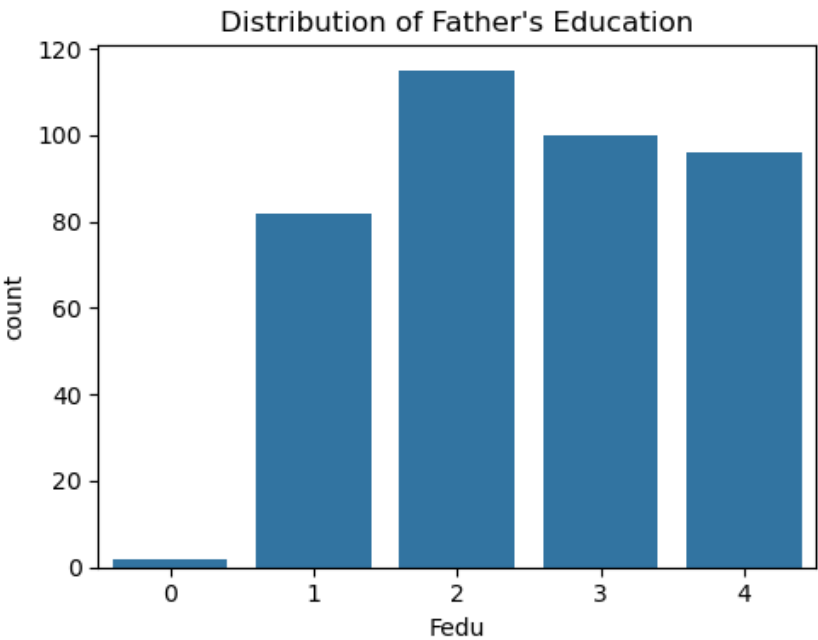
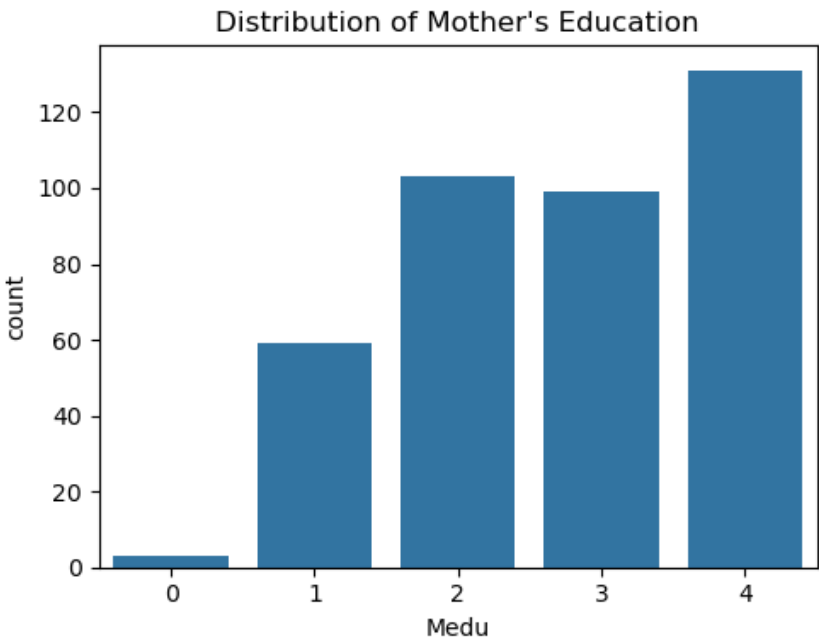
print(Medu.to_string(index=False))
print('\n')
print(Fedu.to_string(index=False))
print('\n')

#Create a countplot for Medu and Fedu
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
sns.countplot(x='Medu', data=studentmath, ax=axes[0])
axes[0].set_title("Distribution of Mother's Education")

sns.countplot(x='Fedu', data=studentmath, ax=axes[1])
axes[1].set_title("Distribution of Father's Education")
plt.tight_layout()
plt.show()
print('')
```

Medu	Average G3
0	13.00
1	8.68
2	9.73
3	10.30
4	11.76

Fedu	Average G3
0	13.00
1	9.16
2	10.26
3	10.66
4	11.36



Preliminary Results for Medu and Fedu

Counterintuitively, students who's parents have an education level of 0 attained the highest average scores. A countplot of these features shows that there are very few examples for parents with 0 education which is leading to instability. I will combine education level 0 with education level 1 to resolve this large imbalance without losing data points. Then I will make a plot of the parental education level vs average G3 to observe the resulting trend.

```
In [249]: print('\n')

#Create copy of the dataset
filtered = studentmath.copy()

#Merge education level 0 with education level 1
filtered['Medu'] = filtered['Medu'].replace(0,1)

#Group by Medu and calculate the mean G3
grouped = filtered.groupby('Medu')['G3'].mean()

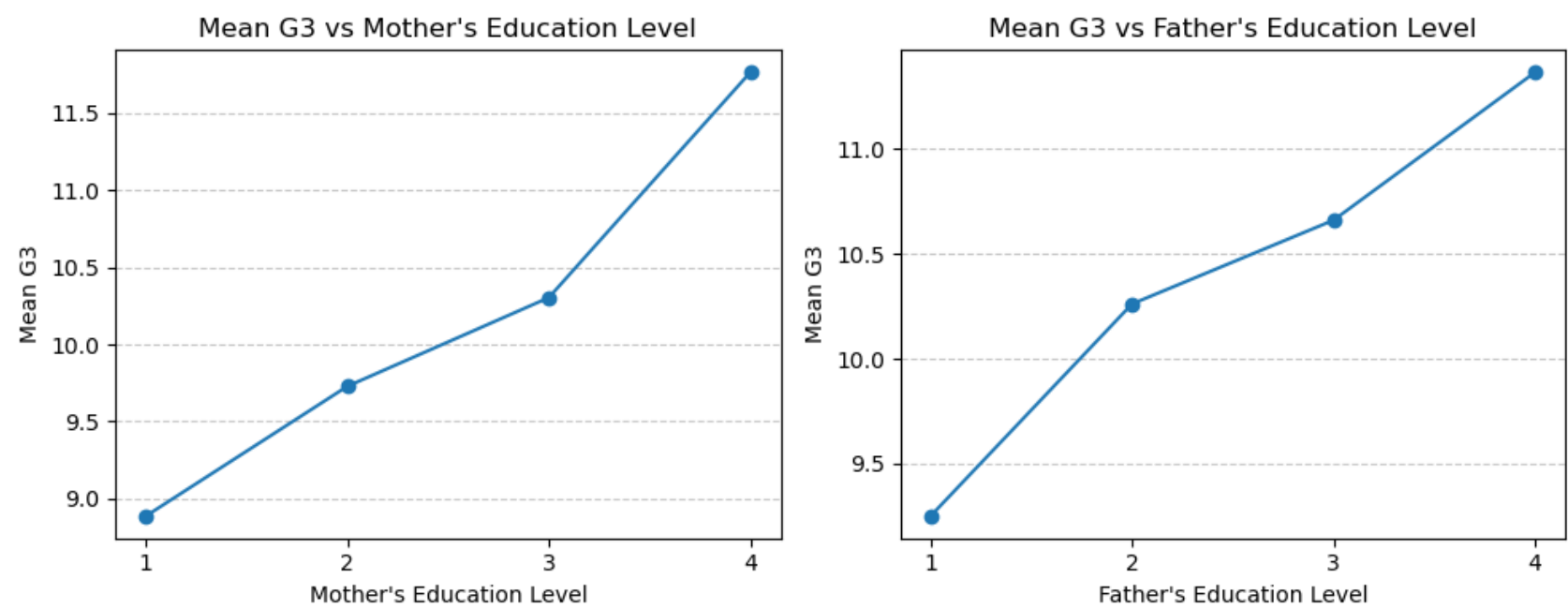
#Set up plot with 2 axes
fig, axes = plt.subplots(1, 2, figsize=(10, 4))

#Plot mother's educatoin vs mean G3 on left axes(axes[0])
grouped.plot(marker='o', ax=axes[0])
axes[0].set_title("Mean G3 vs Mother's Education Level")
axes[0].set_ylabel("Mean G3")
axes[0].set_xlabel("Mother's Education Level")
axes[0].set_xticks([1, 2, 3, 4])
axes[0].grid(axis='y', linestyle='--', alpha=0.7)

filtered['Fedu'] = filtered['Fedu'].replace(0,1)
grouped2 = filtered.groupby('Fedu')['G3'].mean()

#Plot father's educatoin vs mean G3 on right axes(axes[1])
grouped2.plot(marker='o', ax=axes[1])
axes[1].set_title("Mean G3 vs Father's Education Level")
axes[1].set_ylabel("Mean G3")
axes[1].set_xlabel("Father's Education Level")
axes[1].set_xticks([1, 2, 3, 4])
axes[1].grid(axis='y', linestyle='--', alpha=0.7)

#Ensure plots don't overlap
plt.tight_layout()
plt.show()
print('\n')
```



Final Analysis of Medu and Fedu

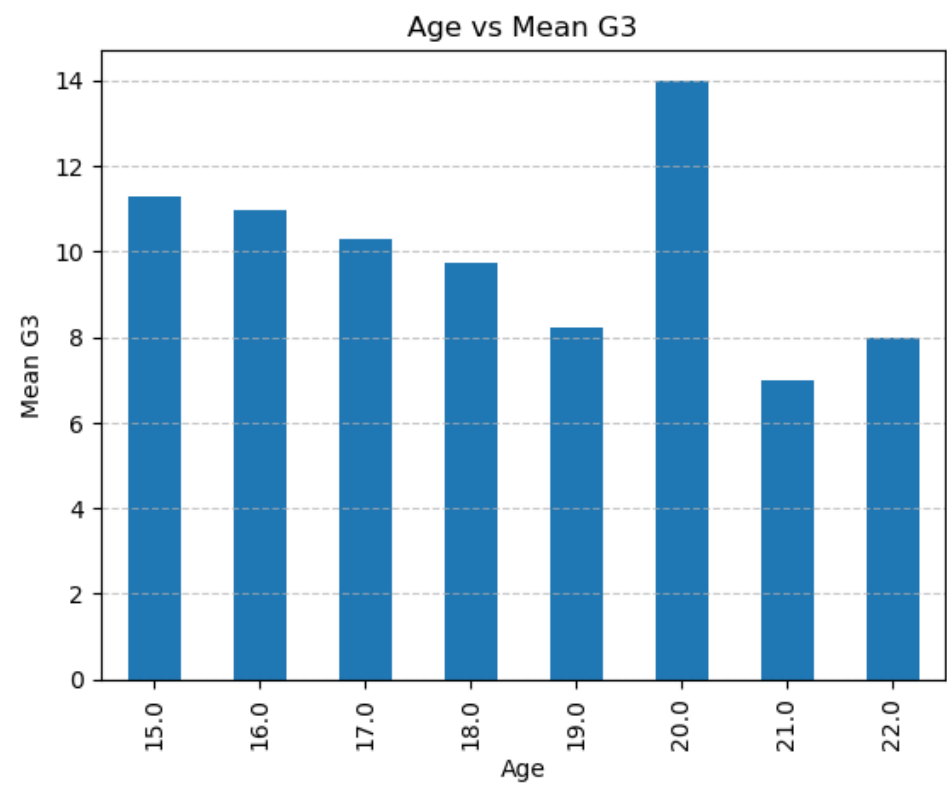
These plots reveal that parental education has a clear, consistent trend once level 0 is merged with level 1 to remove instability due to sparse examples at level 0. Parental education appears to be a useful feature with predictive signal. Medu and Fedu will be included in modeling

Exploration of Age Feature

Next I will examine age which surprisingly has a negative correlation with G3. I will do this by creating some bar plots that show the distribution of student's ages and the average final score for each age group.

```
In [324]: print('')
#Create a bar graph of age vs mean G3 since correlatoin is surprisingly negative
studentmath.groupby('age')['G3'].mean().plot(kind='bar')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.title('Age vs Mean G3')
plt.ylabel('Mean G3')
plt.xlabel('Age')
plt.show()
print(' ')

#Create a table displaying the number of students at each age
age = studentmath['age'].value_counts().reset_index()
age.columns = ['Age', 'Count']
print(age.to_string(index=False))
print(' ')
```



Age	Count
16.0	99
17.0	97
15.0	80
18.0	78
19.0	24
20.0	3
22.0	1
21.0	1

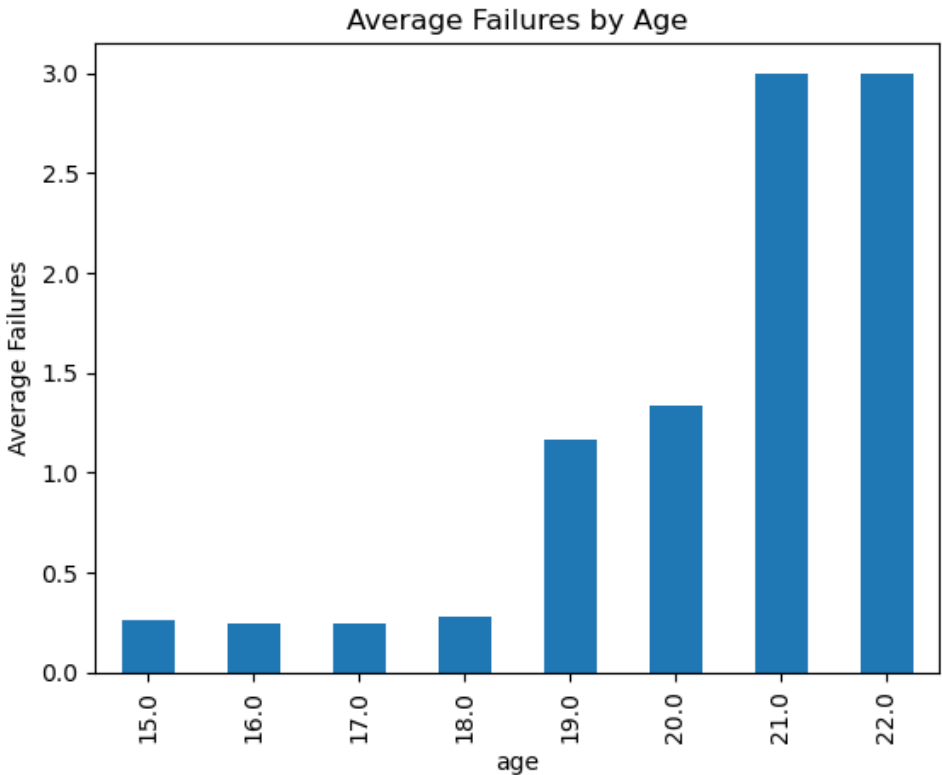
Preliminary Results for Age

Age appears to be negatively correlated with G3 because older students are performing worse on average, however there is some instability due to sparse examples after age 19. Since it is logically unlikely that age itself is causing lower grades I will investigate a possible relationship between age and failures. It is entirely possible that age is capturing signal from failures as students are held back.

```
In [325]: print('\n')
#Investigate possible relationship between age and failures
studentmath.groupby('age')['failures'].mean().plot(kind='bar')
plt.title("Average Failures by Age")
```



```
plt.ylabel("Average Failures")
plt.show()
print('\n')
```



Final Interpretation of Age Feature

Further analysis reveals that the negative correlation between age and G3 is not a direct effect. Older students have significantly more failures. This indicates that age is capturing signal from failures as students are held back. Age is also quite noisy due to sparse instances after age 19. Failures exhibits a clear signal with less noise so age is going to be excluded from modeling.

Exploration of Goout Feature

Next I will examine the relationship between how often students go out with friends and their final grade using some simple tables.

```
In [251]: print('\n')

#Create a table of mean G3 grouped by how often they go out
goout = studentmath.groupby('goout')['G3'].mean().round(2).reset_index()
goout.columns = ['Go Out', 'G3']
print(goout.to_string(index=False))
print(' \n')

#Create a table showing the distribution across the levels of going out
goout2 = studentmath['goout'].value_counts().reset_index()
goout2.columns = ['Go Out', 'Count']
print(goout2.to_string(index=False))
print('\n')
```

Go Out	G3
1	9.87
2	11.19
3	10.96
4	9.65
5	9.04

Go Out	Count
3	130
2	103
4	86
5	53
1	23

Interpretation of Goout Feature

How often students go out appears to exhibit a non-linear relationship with G3. Those who rarely go out with friends perform worse than those who go out a moderate amount. Those who go out extremely often perform the worst. All levels have more than 20 instances so instability does not appear to be a major issue. Goout is likely a useful feature and will be included in modeling.

Exploration of Studytime Feature

Next I will explore how much time students spend studying by creating a line graph of avg G3 vs studytime and a table displaying the count of students at each level of studying

```
In [326]: print('\n')

#Create plot of studytime vs avg G3
#Group by studytime and take the mean of G3 to make a plot
studentmath.groupby('studytime')['G3'].mean().plot(marker='o')
plt.title("Study Time vs Avg Grade (G3)")
plt.ylabel("Avg Grade")
plt.xlabel("Study Time")

#Scale y-axis to show variation clearly
plt.ylim(9, 12)

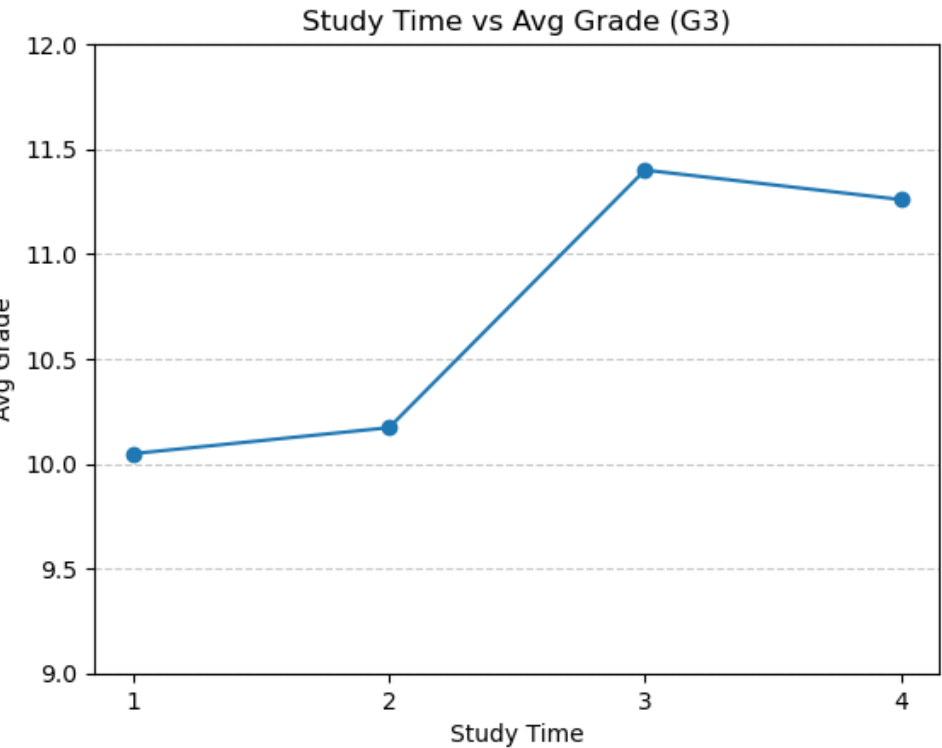
#Set y-axis and x-axis ticks
plt.yticks([9.0, 9.5, 10, 10.5, 11.0, 11.5, 12.0])
```

```
plt.xticks([1,2,3,4])

#Add gridlines to the graph
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()

print('\n')

#Create a table counting how many students are at each level of studying
studytime = studentmath['studytime'].value_counts().reset_index()
studytime.columns = ['Study Time', 'Count']
print(studytime.to_string(index=False))
print('\n')
```



Study Time	Count
2	198
1	105
3	65
4	27

Interpretation of Studytime Feature

The trend for studytime is intuitive and unsurprising. Those who study the least have the worst academic performance and those who study at a level of 3 and 4 perform significantly better. All levels have over 20 instances making this a robust and useful feature with clear signal. Studytime will be included in modeling.

Exploration of Health Feature

Next I will assess the relationship between a student's health and G3 by creating a couple of simple tables.

In [327]

```
print('\n')

#Create table of avg G3 score grouped by health rating
health = studentmath.groupby('health')['G3'].mean().round(2).reset_index()
health.columns = ['Health', 'G3']
print(health.to_string(index=False))

print('\n')

#Create a table displaying how many students are at each health rating
health2 = studentmath['health'].value_counts().reset_index()
health2.columns = ['Health', 'Count']
print(health2.to_string(index=False))

print('\n')
```

Health	G3
1	11.87
2	10.22
3	10.01
4	10.11
5	10.40

Health	Count
5	146
3	91
4	66
1	47
2	45

Interpretation of Health Feature

Health appears to have very minimal separation between the levels of 2,3, 4 and 5 with those who have poor health surprisingly performing the best. Health appears to have a very weak and inconsistent signal. The subjective nature of this self reported feature is most likely creating a lot of noise. The initial correlation with G3 as displayed on the heatmap is also very weak. Since health appears to be low signal and high noise it will excluded from modeling.

Exploration of Alcohol Consumption Features

Next I will explore the alcohol related features by creating tables that illustrate the relationship between these features and G3 as well as the distribution for each of these features.

In [328]

```
print('\n')

#Create a table of avg G3 score grouped by daily alcohol consumption
dalc = studentmath.groupby('Dalc')['G3'].mean().round(2).reset_index()
dalc.columns = ['Dalc', 'Avg G3']
print(dalc.to_string(index=False))

print('\n')

#Create a table of avg G3 score grouped by weekly alcohol consumption
walc = studentmath.groupby('Walc')['G3'].mean().round(2).reset_index()
walc.columns = ['Walc', 'Avg G3']
print(walc.to_string(index=False))

print('\n')

#Create tables displaying the number of students at each level of alcohol consumption
dcount = studentmath['Dalc'].value_counts().reset_index()
dcount.columns = ['Dalc', 'Count']
print(dcount.to_string(index=False))
print('\n')

wcount = studentmath['Walc'].value_counts().reset_index()
wcount.columns = ['Walc', 'Count']
print(wcount.to_string(index=False))
print('\n')
```

Dalc	Avg G3
1	10.73
2	9.25
3	10.50
4	9.89
5	10.67

Walc	Avg G3
1	10.74
2	10.08
3	10.72
4	9.69
5	10.14

Dalc	Count
1	276
2	75
3	26
5	9
4	9

Walc	Count
1	151
2	85
3	80
4	51
5	28

Interpretation of Alcohol Consumption Features

These tables reveal that the alcohol consumption variables appear to be extremely noisy and exhibit no clear pattern in relation to G3. The distribution between the groups is also very imbalanced. Dalc has sparse examples for levels 4 and 5 leading to instability. The alcohol consumption variables appear to be mostly noise with no meaningful signal. Most likely due to being self reported and having very imbalanced groupings leading to instability in the data. The alcohol consumption variables do not appear to be useful and will not be included in modeling.

Exploration of the Family Relationship Feature

Next, I will explore the family relationships feature. I will plot a graph of relationship quality vs average final grade as well as a table illustrating the distribution for this feature.

In [256]

```
print('\n')

#Create a plot of average G3 grouped by family relationship rating
studentmath.groupby('famrel')['G3'].mean().round(2).plot(marker='o')
plt.title("Family Relationship Quality vs Avg Grade (G3)")
plt.ylabel("Avg Grade")
plt.xlabel("Family Relationship Quality")

#Set x-axis ticks
plt.xticks([1,2,3,4, 5])

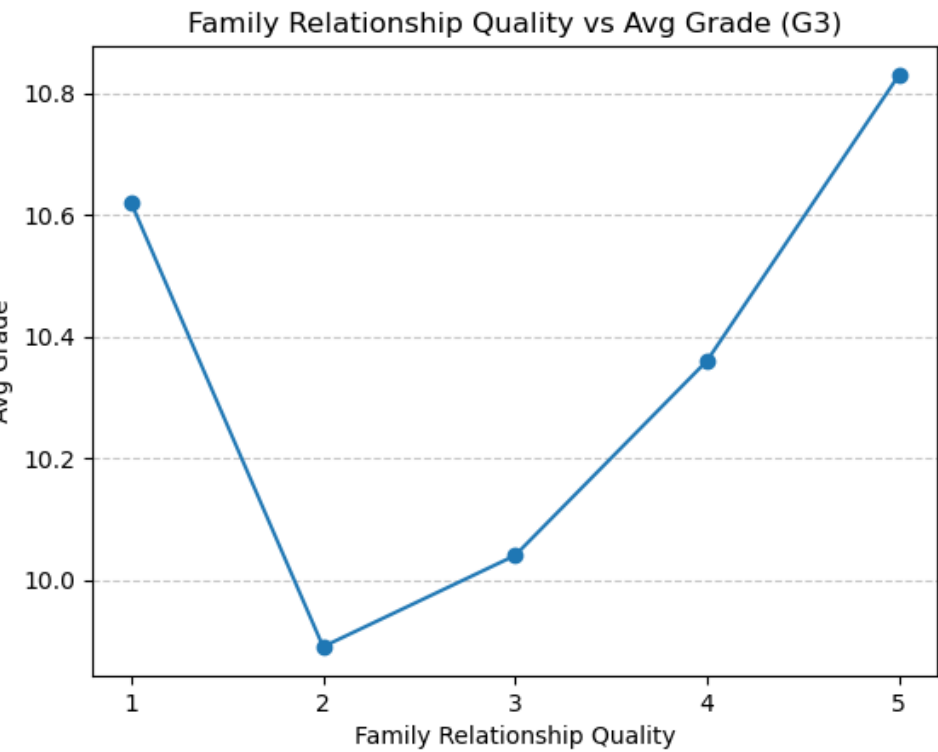
#Add grid lines to the plot
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()

print('\n')

#Create a table displaying the number of students at each quality level
famcount = studentmath['famrel'].value_counts().reset_index()
```

```
famcount.columns = ['Quality', 'Count']
print(famcount.to_string(index=False))

print('\n')
```



Quality	Count
4	195
5	106
3	68
2	18
1	8

Initial Analysis of the Family Relationship Feature

Counterintuitively, those students with the worst family relationships are performing very well compared to the other groups. A table which counts how many students reported each relationship level reveals sparse examples for students reporting a relationship quality of 1. This is very likely leading to instability in the data. I will combine the levels of 1 and 2 to in an attempt to resolve the imbalance issue much like I did previously with the education features and observe a new plot.

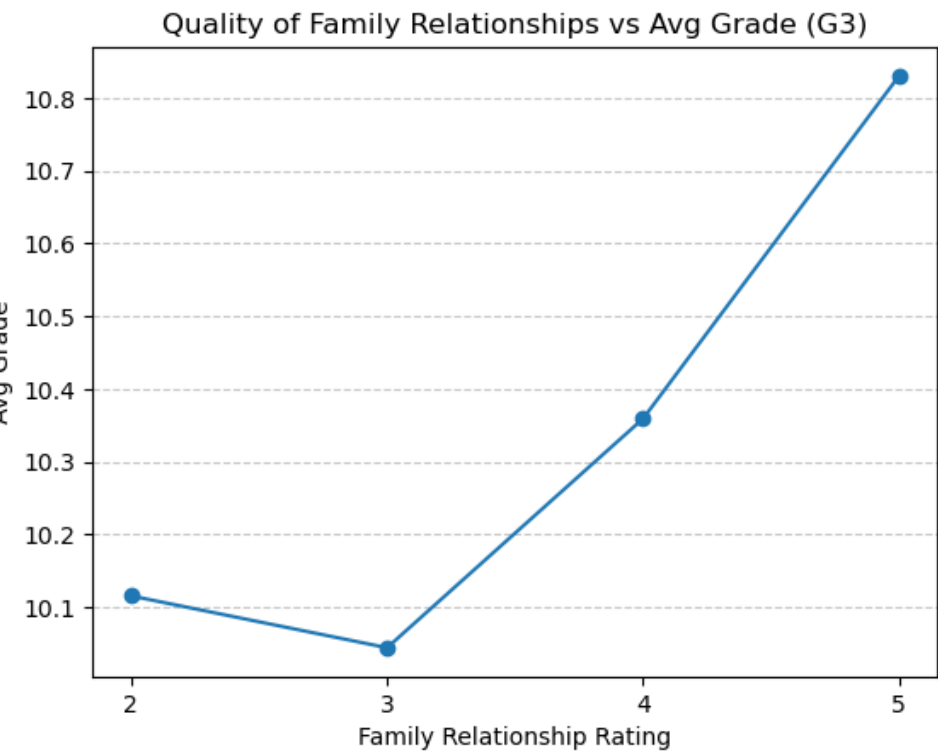
```
In [329]: print('\n')

#Create a dataframe with relationship levels of 1 and 2 combined.
famfiltered = studentmath.copy()
famfiltered['famrel'] = studentmath['famrel'].replace(1,2)

#Calculate average G3 grouped by quality of relationships
famgrouped = famfiltered.groupby('famrel')['G3'].mean()

#Plot quality of family relationships vs mean G3
famgrouped.plot(marker='o')
plt.title("Quality of Family Relationships vs Avg Grade (G3)")
plt.ylabel("Avg Grade")
plt.xlabel("Family Relationship Rating")
plt.xticks([2,3,4, 5])
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()

print('\n')
```



Final Interpretation of the Family Relationship Feature

When family relationship ratings of 1 and 2 are combined the trend becomes much clearer. There appears to be a relatively linear relationship between the quality of family relationships and G3. The correlation is very weak but it may prove to be a useful feature for modeling. Famrel will be included in

modeling.

Exploration of Freetime Feature

The feature, freetime has a correlation of 0.00 with G3 indicating no monotonic relationship. However, a non-linear relationship still needs to be ruled out. I will briefly explore freetime with some tables.

```
In [258]: print('')
free = studentmath.groupby('freetime')['G3'].mean().round(2).reset_index()
free.columns = ['freetime', 'Avg G3']
print(free.to_string(index=False))
print(' ')

freecounts = studentmath['freetime'].value_counts().reset_index()
freecounts.columns = ['freetime', 'Count']
print(freecounts.to_string(index=False))
print('')

freetime  Avg G3
         1    9.84
         2   11.56
         3    9.78
         4   10.43
         5   11.30

freetime  Count
         3    157
         4    115
         2     64
         5     40
         1     19
```

Interpretation of Freetime

This analysis of freetime indicates that there is no meaningful relationship between freetime and G3. The feature is very noisy with instability due to large imbalances and a correlation of 0.00 with G3. Freetime will be excluded from modeling.

Exploration of Yes/No Categorical Features

Next, I will begin to analyze the categorical features starting with the yes/no, binary features. The primary method of analysis will be calculating the separation across the categories of yes and no which is an indication of the feature's practical signal strength.

```
In [243]: print('\n')
print('\n')

#Create a list of yes, no features based off the codebook
yes_no_features = ['schoolsup', 'famsup', 'paid', 'activities', 'nursery', 'higher', 'internet', 'romantic']

#Create an empty list for results
results = []

#For each feature, find the mean G3 grouped by category and append that to the results list
#Also create a column which calculates the separation between the choices
for col in yes_no_features:
    grouped = studentmath.groupby(col)['G3'].mean().round(2)

    results.append({
        'FEATURE': col,
        'NO': grouped.get('no', None),
        'YES': grouped.get('yes', None),
        'Separation': abs(grouped['yes'] - grouped['no'])
    })

#Convert the results list into an organized dataframe
results_df = pd.DataFrame(results)

#Sort the dataframe features by separation in ascending order
results_df = results_df.sort_values(by='Separation', ascending = False)

#Print results without index labels
print(results_df.to_string(index=False))
print('')
print('')
```

FEATURE	NO	YES	Separation
higher	6.80	10.61	3.81
romantic	10.84	9.58	1.26
internet	9.41	10.62	1.21
schoolsup	10.56	9.43	1.13
paid	9.99	10.92	0.93
nursery	9.95	10.54	0.59
famsup	10.64	10.27	0.37
activities	10.34	10.49	0.15

Initial Analysis of Yes/No Binary Features

Analysis of these features indicates that nursery, famsup and activities show minimal separation between their categories which suggests that they have minimal predictive value. Features with very little separation between categories are unlikely to provide strong predictive signal. Nursery, famsup and activities will be excluded from modeling.

Investigation of Negative Relationship for Schoolsup

Students receiving school support having worse performance is most likely due to a reverse correlation with G3 so I will explore this feature further.

In [330]

```
print('\n')

#Calculate mean G1 and G2 grouped by school support
print(studentmath.groupby('schoolsup')[['G1', 'G2']].mean().round(2))

print('\n')
```

	G1	G2
schoolsup		
no	11.18	10.88
yes	9.08	9.57

Final Interpretation of Schoolsup Feature

This table suggests that students who are receiving school support have had poorer previous academic performance. This indicates that schoolsup is most likely an intervention for students who are already struggling. This suggests that the feature likely acts as a proxy for prior academic difficulty which should have useful predictive power. Schoolsup will be included in modeling.

Exploration of Remaining Categorical Features

Next, the remaining categorical features which are not Yes/No will be analysed in a similar way. I will create a table illustrating the separation between their categories.

In [263]

```
print('\n')

#Create a list of the remaining categorical features
categorical_features = ['school', 'sex', 'address', 'famsize', 'Pstatus',
                        'Mjob', 'Fjob', 'reason', 'guardian']

#Create an empty list storing grouped results
results = []

#Calculate the mean G3 for each category within each feature
for col in categorical_features:
    grouped = studentmath.groupby(col)['G3'].mean().round(2).reset_index()

    #Name the two existing table columns for readability
    grouped.columns = ['CATEGORY', 'AVG G3']

    #Create a table column displaying the name of each feature
    grouped['FEATURE'] = col

    #Append grouped result to the results list
    results.append(grouped)

#Calculate the separation between the highest and lowest category means
separation = grouped['AVG G3'].max() - grouped['AVG G3'].min()

#Add a row that displays the separation for each feature
results.append(pd.DataFrame([{'FEATURE': 'Separation', 'CATEGORY': '',
                              'AVG G3': round(separation, 2)}]))

#Add a blank row between features for readability
results.append(pd.DataFrame([{'FEATURE': '', 'CATEGORY': '',
                              'AVG G3': ''}]))

#Combine all grouped results into a single dataframe
cat_df = pd.concat(results, ignore_index = True)

#Arrange columns into the desired order
cat_df = cat_df[['FEATURE', 'CATEGORY', 'AVG G3']]

#Print the combined dataframe without index labels
print(cat_df.to_string(index=False))
print('\n')
```

FEATURE	CATEGORY	AVG	G3
school	GP	10.49	
school	MS	9.85	
Separation		0.64	
sex	F	9.97	
sex	M	10.91	
Separation		0.94	
address	R	9.51	
address	U	10.67	
Separation		1.16	
famsize	GT3	10.18	
famsize	LE3	11.0	
Separation		0.82	
Pstatus	A	11.2	
Pstatus	T	10.32	
Separation		0.88	
Mjob	at_home	9.15	
Mjob	health	12.15	
Mjob	other	9.82	
Mjob	services	11.02	
Mjob	teacher	11.05	
Separation		3.0	
Fjob	at_home	10.15	
Fjob	health	11.61	
Fjob	other	10.19	
Fjob	services	10.3	
Fjob	teacher	11.97	
Separation		1.82	
reason	course	9.82	
reason	home	10.26	
reason	other	11.17	
reason	reputation	11.14	
Separation		1.35	
guardian	father	10.69	
guardian	mother	10.48	
guardian	other	9.06	
Separation		1.63	

Initial Interpretation of the Categorical Features

This table indicates that aside from school, the remaining categorical features demonstrate meaningful separation across categories and will likely have useful predictive signal. Based on these results, all of the remaining features will be retained for modeling except for school.

Exploration of Possible Redundancy between Parental Features

Next, I will explore the relationship between parental education and parental occupation to determine if there is potential redundancy and multicollinearity between these parental features.

In [280]:

```
print('\n')

#Create a plot showing the distribution of Mother's jobs in relation to their education level.
plt.figure(figsize=(10,6))

sns.countplot(
    data=studentmath,
    x='Medu',
    hue='Mjob'
)

plt.title("Distribution of Mother's Jobs by Education Level")
plt.xlabel("Mother's Education Level")
plt.ylabel("Count")
plt.show()

print('\n')

#Create a plot showing the distribution of Father's jobs in relation to their education level
plt.figure(figsize=(10,6))

sns.countplot(
    data=studentmath,
    x='Fedu',
    hue='Fjob' )

plt.title("Distribution of Father's Jobs by Education Level")
plt.xlabel("Father's Education Level")
plt.ylabel("Count")
plt.show()

print('\n')

#Create a table which groups jobs by education level and calculates the mean G3 for each
medu_job = studentmath.groupby(['Medu', 'Mjob'])['G3'].mean().reset_index().round(2)
medu_job.columns = ['Medu Level', 'Mjob', 'Avg G3']

print(medu_job.to_string(index = False))

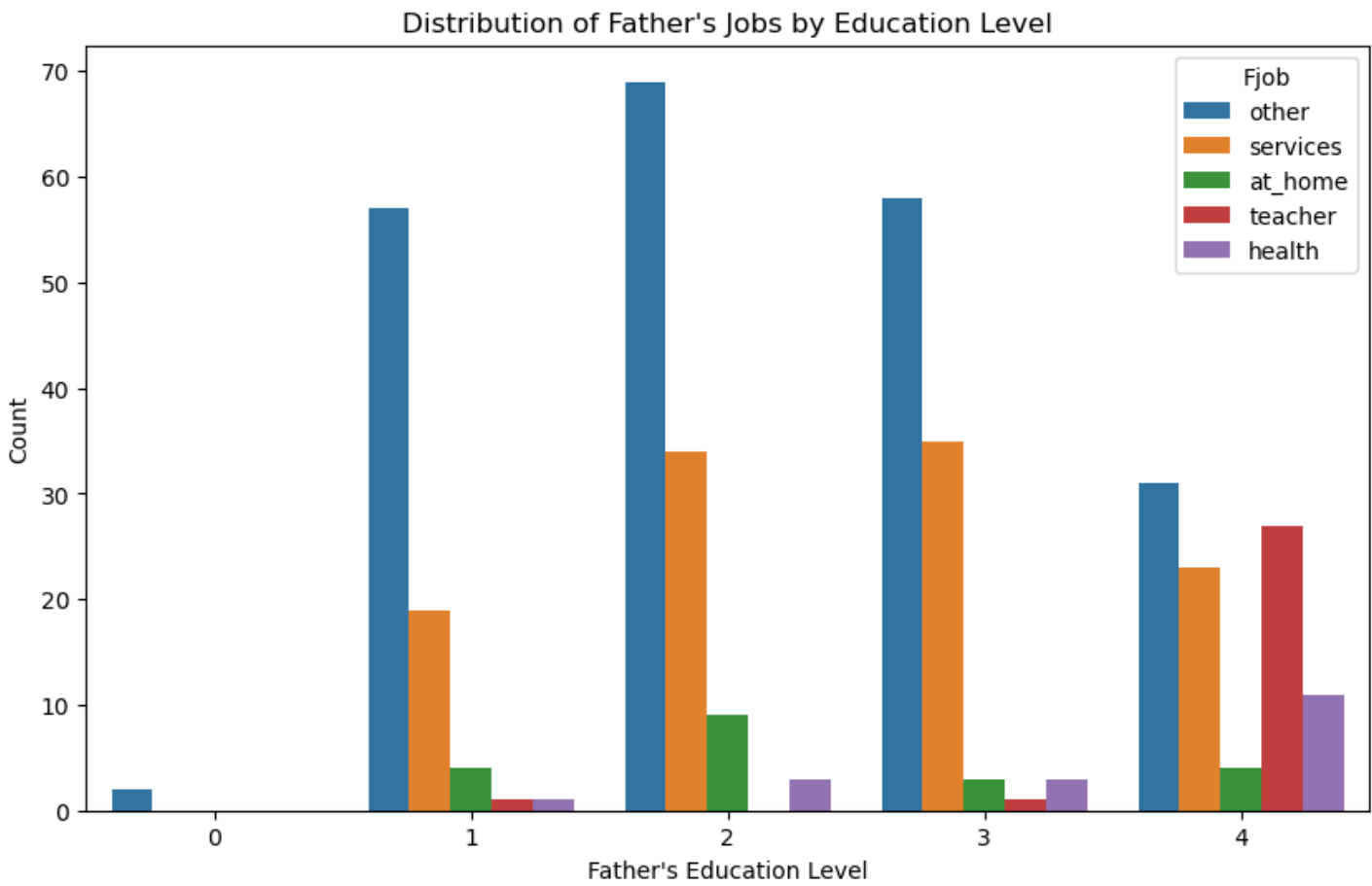
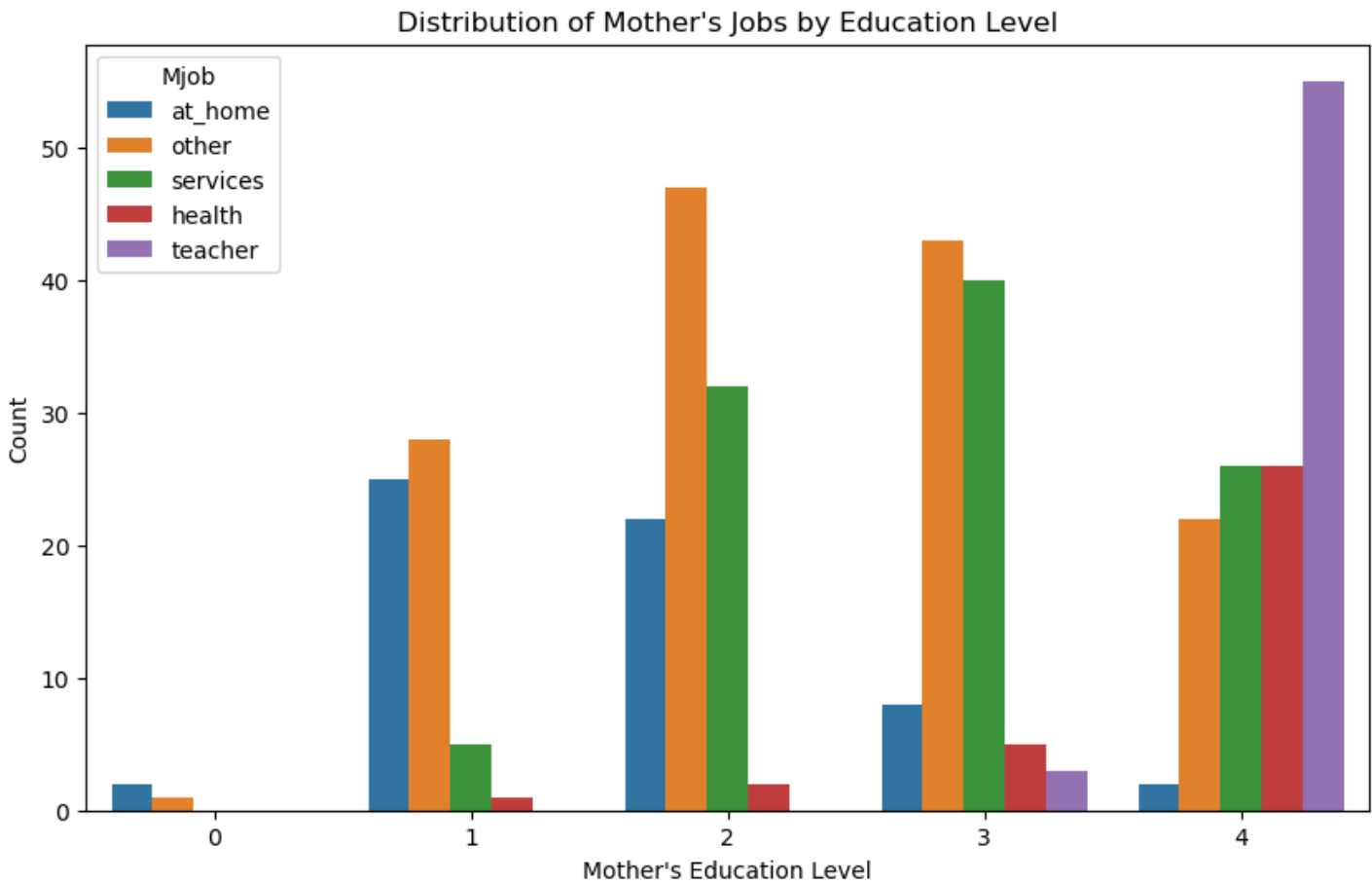
print('\n')

#Create a table which groups jobs by education level and calculates the mean G3 for each
fedu_job = studentmath.groupby(['Fedu', 'Fjob'])['G3'].mean().reset_index().round(2)
```

```
fedu_job.columns = ['Fedu Level', 'Fjob', 'Avg G3']

print(fedu_job.to_string(index = False))

print('\n')
```



Medu	Level	Mjob	Avg G3
0		at_home	12.00
0		other	15.00
1		at_home	9.68
1		health	8.00
1		other	7.71
1		services	9.20
2		at_home	9.05
2		health	8.50
2		other	9.72
2		services	10.28
3		at_home	6.75
3		health	10.20
3		other	10.63
3		services	10.75
3		teacher	9.33
4		at_home	10.50
4		health	12.96
4		other	10.91
4		services	12.69
4		teacher	11.15

Fedu	Level	Fjob	Avg G3
0		other	13.00
1		at_home	12.00
1		health	13.00
1		other	8.77
1		services	9.37
1		teacher	12.00
2		at_home	9.67
2		health	14.00
2		other	10.52
2		services	9.56
3		at_home	11.67
3		health	9.33
3		other	10.36
3		services	11.00
3		teacher	17.00
4		at_home	8.25
4		health	11.45
4		other	11.58
4		services	11.09
4		teacher	11.78

Final Interpretation of Parental Features

This analysis indicates that teachers and healthcare workers are more prevalent among those parents who have education levels of 3 and 4. It also reveals that staying at home is more prevalent among those with less education. Those who are more educated and hold the same job category also appear to have higher performing children. However, the grouped averages appear to reveal that parental occupation still provides separation within each education level. Although parental education is partially correlated with parental occupation, the variables still appear to provide some additional predictive signal. Mjob and Fjob will be included in modeling.

Custom Transformer Creation

Next, I will create a custom transformer which performs some basic feature engineering and drops unused columns. It will combine all 3 absence columns into a total absence column and drop the original redundant columns. It will also merge parental education levels of 0 and 1 to resolve the imbalance in those features without losing data points. It will similarly merge family relationship levels of 1 and 2 to resolve the imbalance in that feature as well. The features of G1 and G2 will be retained so that they might be used in later model analysis if desired.

```
In [6]: from sklearn.base import BaseEstimator, TransformerMixin

#Initialize custom transformer class
class Transformer(BaseEstimator, TransformerMixin):
    def __init__(self):
        pass

    #Define the fit method for this custom transformer
    def fit(self, X, y=None):
        return self

    #Define transform method for this transformer
    def transform(self, X):

        #Create a copy to avoid modifying the original dataset
        X = X.copy()

        #Create a feature which is the sum of all absence columns
        X['total_absences'] = (X['absences_G1'] + X['absences_G2'] + X['absences_G3'])

        #Drop the now redundant absence columns after feature engineering
        X = X.drop(columns = ['absences_G1', 'absences_G2', 'absences_G3'])

        #Drop all features which are unlikely to have useful predictive signal based on prior analysis
        X = X.drop(columns = ['freetime', 'age', 'health', 'Dalc', 'Walc', 'nursery', 'famsup',
                               'activities', 'school'])

        #Merge the sparse parental education level of 0 with level 1 to alleviate instability
        X['Medu'] = X['Medu'].replace(0,1)
        X['Fedu'] = X['Fedu'].replace(0,1)

        #Merge sparse relationship level of 1 with level 2 to alleviate instability
        X['famrel'] = X['famrel'].replace(1,2)

        return X
```

Creation of Column Lists for Pipelines

In this step I will create a list of numeric, categorical and ordinal columns to be used in pipelines

```
In [7]: #Create a list for each feature type used by the pipeline
numeric_columns = ['total_absences']

categorical_columns = ['higher', 'romantic', 'internet', 'schoolsup', 'guardian', 'paid', 'sex', 'address', 'famsize',
                      'Pstatus', 'reason', 'Mjob', 'Fjob']

ordinal_columns = ['Medu', 'Fedu', 'failures', 'studytime', 'traveltime', 'goout', 'famrel']
```

Creation of Preprocessing Pipelines

In this step I will create a numeric, categorical and ordinal preprocessing pipeline for the various feature types as well as a full pipeline which incorporates the custom transformer and combines all of the feature specific pipelines. Then I will fit and transform the training data.

```
In [8]: from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import OneHotEncoder
from sklearn.preprocessing import OrdinalEncoder
from sklearn.compose import ColumnTransformer

#Create numeric preprocessing pipeline which imputes missing values and scales the features
numeric_pipeline = make_pipeline(
    SimpleImputer(strategy='median'),
    StandardScaler())

#Create categorical preprocessing pipeline which imputes missing values and performs one hot encoding
categorical_pipeline = make_pipeline(
    SimpleImputer(strategy = 'most_frequent'),
    OneHotEncoder(drop='first', sparse_output=False) )

#Create ordinal preprocessing pipeline which imputes missing values and performs ordinal encoding
ordinal_pipeline = make_pipeline(
    SimpleImputer(strategy='most_frequent'),
    OrdinalEncoder() )

#Create full preprocessing pipeline which combines the custom transformer with column specific pipelines
full_pipeline = make_pipeline(
    Transformer(),
    ColumnTransformer([
        ('num', numeric_pipeline, numeric_columns),
        ('cat', categorical_pipeline, categorical_columns),
        ('ord', ordinal_pipeline, ordinal_columns) ]))

#Fit the preprocessing pipline and transform the training data
X_train_transformed = full_pipeline.fit_transform(X_train)
```

Inspection of Transformed Dataset

Next, it is important to inspect the fit and transformed dataset to ensure that each component of the pipeline is functioning properly.

```
In [30]: from IPython.display import display

print('\n')

#Verify dimensions of the transformed dataset
print('Shape: ', X_train_transformed.shape)
print('\n')

#Verify the number of missing values is zero
print('Missing Values: ', np.isnan(X_train_transformed).sum())
print('\n')

#Create a dataframe from X_train_transformed with feature names intact
feature_names = full_pipeline.named_steps['columntransformer'].get_feature_names_out()
df_train = pd.DataFrame(X_train_transformed, columns=feature_names)

#Transpose the dataframe to view features vertically
display(df_train.head(8).T)
print('\n')
```

Shape: (316, 30)

Missing Values: 0

	0	1	2	3	4	5	6	7
num__total_absences	-0.571127	-0.84793	-0.84793	-0.571127	-0.84793	1.089696	-0.017519	-0.84793
cat__higher_yes	1.000000	1.00000	1.00000	1.000000	1.00000	1.000000	1.000000	1.00000
cat__romantic_yes	1.000000	0.00000	1.00000	0.000000	0.00000	1.000000	1.000000	0.00000
cat__internet_yes	1.000000	1.00000	1.00000	1.000000	0.00000	1.000000	1.000000	0.00000
cat__schoolsup_yes	0.000000	0.00000	0.00000	1.000000	0.00000	0.000000	0.000000	0.00000
cat__guardian_mother	1.000000	0.00000	1.00000	1.000000	1.00000	1.000000	0.000000	1.00000
cat__guardian_other	0.000000	0.00000	0.00000	0.000000	0.00000	0.000000	0.000000	0.00000
cat__paid_yes	1.000000	0.00000	0.00000	1.000000	0.00000	0.000000	0.000000	0.00000
cat__sex_M	1.000000	1.00000	0.00000	0.000000	1.00000	0.000000	0.000000	1.00000
cat__address_U	1.000000	1.00000	1.00000	1.000000	0.00000	0.000000	0.000000	0.00000
cat__famsize_LE3	0.000000	0.00000	0.00000	0.000000	0.00000	0.000000	0.000000	0.00000
cat__Pstatus_T	1.000000	1.00000	1.00000	1.000000	1.00000	1.000000	1.000000	1.00000
cat__reason_home	1.000000	1.00000	1.00000	1.000000	0.00000	0.000000	0.000000	0.00000
cat__reason_other	0.000000	0.00000	0.00000	0.000000	0.00000	0.000000	0.000000	1.00000
cat__reason_reputation	0.000000	0.00000	0.00000	0.000000	0.00000	1.000000	0.000000	0.00000
cat__Mjob_health	0.000000	0.00000	0.00000	0.000000	0.00000	0.000000	0.000000	0.00000
cat__Mjob_other	0.000000	1.00000	0.00000	0.000000	1.00000	1.000000	0.000000	1.00000
cat__Mjob_services	1.000000	0.00000	0.00000	0.000000	0.00000	0.000000	0.000000	0.00000
cat__Mjob_teacher	0.000000	0.00000	0.00000	1.000000	0.00000	0.000000	0.000000	0.00000
cat__Fjob_health	0.000000	0.00000	0.00000	1.000000	0.00000	0.000000	0.000000	0.00000
cat__Fjob_other	1.000000	1.00000	0.00000	0.000000	1.00000	1.000000	1.000000	0.00000
cat__Fjob_services	0.000000	0.00000	1.00000	0.000000	0.00000	0.000000	0.000000	1.00000
cat__Fjob_teacher	0.000000	0.00000	0.00000	0.000000	0.00000	0.000000	0.000000	0.00000
ord__Medu	2.000000	1.00000	0.00000	3.000000	1.00000	2.000000	1.000000	0.00000
ord__Fedu	2.000000	2.00000	2.00000	2.000000	0.00000	0.000000	3.000000	0.00000
ord__failures	0.000000	0.00000	3.00000	0.000000	0.00000	1.000000	0.000000	1.00000
ord__studytime	1.000000	0.00000	1.00000	2.000000	0.00000	1.000000	2.000000	0.00000
ord__traveltime	0.000000	1.00000	0.00000	0.000000	1.00000	0.000000	0.000000	1.00000
ord__goout	2.000000	2.00000	4.00000	3.000000	1.00000	2.000000	2.000000	1.00000
ord__famrel	3.000000	4.00000	3.00000	2.000000	2.00000	4.000000	3.000000	3.00000

Model Evaluation

The pipeline appears to be functioning properly so it is now time to begin modeling. To begin modeling I will choose five different model types to evaluate using RMSE with cross fold validation. The models that I will be evaluating are: Linear Regression, Support Vector Regression, Lasso Regression, Decision Tree Regression and Gradient Boosting Regression. These models each have different approaches which allows them to be more or less reliable depending on the relationships in the data. Linear Regression is strongest when the relationships are simple and linear for example and Gradient Boosting Regression performs well when the relationships are non-linear and involve interactions between the variables.

```
In [9]: from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LinearRegression
from sklearn.svm import SVR
from sklearn.linear_model import Lasso
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import GradientBoostingRegressor

print('\n')

#Instantiate a Linear Regression Model
lin_reg = LinearRegression()

#Instantiate a Support Vector Regression Model
svm_reg = SVR()

#Instantiate a Lasso Regression model
lasso_reg = Lasso(random_state=42)

#Instantiate a Decision Tree Regression model with a max depth of 6
tree = DecisionTreeRegressor(max_depth=6, random_state=42)

#Instantiate a Gradient Boosting Regression model with basic hyper parameters
gradient = GradientBoostingRegressor(n_estimators=200,
    learning_rate=0.05,
    max_depth=3,
    min_samples_leaf=5,
    subsample=0.8,
    random_state=42)

#Calculate RMSE scores for each regression model using cross validation with 5 folds
cv_scores_linear = -cross_val_score(lin_reg, X_train_transformed, y_train, cv=5, scoring = 'neg_root_mean_squared_error')
cv_scores_svm = -cross_val_score(svm_reg, X_train_transformed, y_train, cv=5, scoring = 'neg_root_mean_squared_error')
cv_scores_lasso = -cross_val_score(lasso_reg, X_train_transformed, y_train, cv=5, scoring = 'neg_root_mean_squared_error')
cv_scores_tree = -cross_val_score(tree, X_train_transformed, y_train, cv=5, scoring = 'neg_root_mean_squared_error')
cv_scores_gradient = -cross_val_score(gradient, X_train_transformed, y_train, cv=5, scoring = 'neg_root_mean_squared_error')

#Calculate the mean RMSE score for each model
rmse_lin = cv_scores_linear.mean().round(2)
rmse_svm = cv_scores_svm.mean().round(2)
```

```
rmse_lasso = cv_scores_lasso.mean().round(2)
rmse_tree = cv_scores_tree.mean().round(2)
rmse_gradient = cv_scores_gradient.mean().round(2)

#Print the RMSE score for each model
print("RMSE Linear", rmse_lin)
print("RMSE SVM", rmse_svm)
print('RMSE Lasso', rmse_lasso)
print('RMSE Tree', rmse_tree)
print('RMSE Gradient', rmse_gradient)
print('\n')
```

RMSE Linear 4.54
RMSE SVM 4.32
RMSE Lasso 4.46
RMSE Tree 4.69
RMSE Gradient 4.2

Model Experimentation

Since the parental occupation features add 8 additional variables which results in a lot of added model complexity, I'm going to evaluate model performance without the parental job features. If model accuracy improves I will trim these features.

```
In [10]: print('\n')

#Remove job features from the categorical columns list
categorical_columns = ['higher', 'romantic', 'internet', 'schoolsup', 'guardian', 'paid', 'sex', 'address', 'famsize',
                      'Pstatus', 'reason']

#Rebuild full pipeline
full_pipeline = make_pipeline(
    Transformer(),
    ColumnTransformer([
        ('num', numeric_pipeline, numeric_columns),
        ('cat', categorical_pipeline, categorical_columns),
        ('ord', ordinal_pipeline, ordinal_columns) ] ) )

#Fit the preprocessing pipline and transform the training data with the new feature list
X_train_transformed2 = full_pipeline.fit_transform(X_train)

#Calculate RMSE scores for each regression model using cross validation with 5 folds again
cv_scores_linear2 = -cross_val_score(lin_reg, X_train_transformed2, y_train, cv=5, scoring = 'neg_root_mean_squared_error')
cv_scores_svm2 = -cross_val_score(svm_reg, X_train_transformed2, y_train, cv=5, scoring = 'neg_root_mean_squared_error')
cv_scores_lasso2 = -cross_val_score(lasso_reg, X_train_transformed2, y_train, cv=5, scoring = 'neg_root_mean_squared_error')
cv_scores_tree2 = -cross_val_score(tree, X_train_transformed2, y_train, cv=5, scoring = 'neg_root_mean_squared_error')
cv_scores_gradient2 = -cross_val_score(gradient, X_train_transformed2, y_train, cv=5, scoring = 'neg_root_mean_squared_error')

#Calculate the mean RMSE score for each model
rmse_lin2 = cv_scores_linear2.mean().round(2)
rmse_svm2 = cv_scores_svm2.mean().round(2)
rmse_lasso2 = cv_scores_lasso2.mean().round(2)
rmse_tree2 = cv_scores_tree2.mean().round(2)
rmse_gradient2 = cv_scores_gradient2.mean().round(2)

#Print the RMSE score for each model
print('RMSE Scores Without Parental Occupation Features \n')
print("RMSE Linear", rmse_lin2)
print("RMSE SVM", rmse_svm2)
print('RMSE Lasso', rmse_lasso2)
print('RMSE Tree', rmse_tree2)
print('RMSE Gradient', rmse_gradient2)
print('\n')
```

RMSE Scores Without Parental Occupation Features

RMSE Linear 4.38
RMSE SVM 4.28
RMSE Lasso 4.46
RMSE Tree 4.59
RMSE Gradient 4.13

Experimentation Result

Removing the parental occupation features improved performance across most of the models with Lasso Regression being the exception. This suggests that these features provide limited additional predictive signal and are largely redundant with other variables, particularly parental education.

Model Selection and Fine-Tuning

The Gradient Boosting Regressor appears to be the most reliable model when making predictions with this dataset. Next, I will perform a Grid Search to fine-tune the Gradient Boosting model and determine its optimal hyper parameters.

```
In [308]: from sklearn.model_selection import GridSearchCV

print('\n')

#Create a dictionary of hyperparameters to test
param_grid = {'n_estimators': [100, 200, 300],
              'learning_rate': [0.01, 0.05, 0.1],
              'max_depth': [2, 3, 4, 5],
              'min_samples_leaf': [1, 2, 3, 5],
              'subsample': [0.7, 0.8, 0.9, 1.0] }
```

```
#Instantiate GridSearch
grid_search = GridSearchCV(GradientBoostingRegressor(random_state=42), param_grid,
                           scoring = 'neg_root_mean_squared_error', cv=5, verbose=1)

#fit grid_search to data
grid_search.fit(X_train_transformed2, y_train)

#Define best parameters and best score
best_params = grid_search.best_params_
best_score = -grid_search.best_score_.round(2)

#Print best parameters and best score
print("Best parameters: ", best_params)
print("Best RMSE: ", best_score)
print('\n')
```

Fitting 5 folds for each of 576 candidates, totalling 2880 fits
Best parameters: {'learning_rate': 0.01, 'max_depth': 4, 'min_samples_leaf': 2, 'n_estimators': 300, 'subsample': 0.9}
Best RMSE: 3.9708039171583756

Fine-Tuning Result

Fine-tuning the Gradient Boosting Regressor's hyper parameters resulted in an RMSE score of 3.97 which is a slight improvement over the previous best score of 4.12. The optimal configuration favored a low learning rate combined with a larger number of estimators and moderate tree depth, indicating that more regularized boosting iterations provided the best performance. This suggests that the relationship between the provided student features and final grade has nonlinear aspects but benefits from controlled model complexity rather than highly flexible trees that may lead to overfitting.

Model Evaluation on Test Set

Next, model performance will be evaluated on the test set using both RMSE and R².

In [309]:

```
from sklearn.metrics import root_mean_squared_error, r2_score

print('\n')

#Define the best model
best_model = GradientBoostingRegressor(learning_rate = 0.01, max_depth = 4, min_samples_leaf = 2, n_estimators = 300,
                                       subsample = 0.9, random_state=42)

#Fit the best model to transformed training data
best_model.fit(X_train_transformed2, y_train)

#Transform the test data using the fitted pipeline
X_test_transformed = full_pipeline.transform(X_test)

#Generate predictions from test data
predictions = best_model.predict(X_test_transformed)

#Calculate RMSE
rmse = root_mean_squared_error(y_test, predictions)

#Calculate R-Squared
r2 = r2_score(y_test, predictions)

print('Test RMSE: ', round(rmse, 2))
print('\n')
print('Test R²: ', round(r2, 2))
print('\n')
```

Test RMSE: 3.86

Test R²: 0.27

Conclusions

The final Gradient Boosting model was able to achieve a test RMSE of 3.86 and an R² score of 0.27. A substantial improvement over the baseline models evaluated earlier in this project. This result indicates that predictions are off by approximately 3.86 points out of 20.

An R² score of 0.27 indicates that this model explains 27% of the variance in student performance with the remaining 73% being attributed to random variation and unknown factors. While this model demonstrates meaningful predictive capability, it falls short of providing actionable decision making predictions on its own.

However, when combined with previous academic performance indicators such as early quiz or test scores, the model may provide practical guidance as a screening tool. In this regard the model may be used to rank students in terms of their relative risk of performing poorly. This risk profile could then be used in conjunction with early scores to identify those who need intervention sooner than using academic scores alone.